MILESTONE 3 • 3 MARKS

Project Design Presentation

JAVA DESKTOP APPLICATION

Swing UI • JDBC • Database

PROJECT TITLE

[Enter project title]

Team No.: [ ] | Class/Semester: [ ]
Review Date: [ ]

DESIGN MUST CONNECT

01 Requirements

02 Architecture

03 Classes + Data

04 Screens + Interactions

Every design element should trace back to an approved requirement.

MANDATORY EVIDENCE

Team and HackerRank Java progress

Compare Milestone 2 with the Java stars visible on the current review date

#	Student name	Roll no.	Role	Profile / username	M2 Java stars	Current Java stars	Change
1	[Name]	[Roll no.]	**Team Lead**	[Profile link]	[0–5]	**[0–5]**	**[+ / −]**
2	[Name]	[Roll no.]	Member	[Profile link]	[0–5]	**[0–5]**	**[+ / −]**
3	[Name]	[Roll no.]	Member	[Profile link]	[0–5]	**[0–5]**	**[+ / −]**
4	[Name]	[Roll no.]	Member	[Profile link]	[0–5]	**[0–5]**	**[+ / −]**
5	[Name]	[Roll no.]	Member	[Profile link]	[0–5]	**[0–5]**	**[+ / −]**

REQUIRED

Include a working profile link or a current screenshot. Explain progress for every team member.

This same table must continue in Code Review 1 and all later submissions.

Approved scope and design changes

Start from the Milestone 2 requirements baseline; record any approved change explicitly

APPROVED DESIGN SCOPE

- [Primary user goal]
- [In-scope modules]
- [Required reports / outputs]
- [Mandatory Java, Swing and JDBC baseline]

Design objective

[One sentence explaining the chosen design approach]

Change ID	Requirement	Change and reason	Status
DC-01	[FR-__]	[No change / describe approved change]	[Approved]
DC-02	[FR-__]	[Describe another change, if any]	[Approved]
DC-03	[FR-__]	[Describe another change, if any]	[Approved]

RULE

If a requirement changes, update the Requirements Document, the Design Document and the PPT together.

Unapproved scope additions should be moved to future enhancements.

Layered desktop application design

Separate Swing event handling, application logic, domain behaviour and JDBC persistence

01 **Presentation layer**
Swing frames, dialogs, panels, event handlers
[UI classes / package]

↓

02 **Application and domain layer**
Use-case coordination, business rules, validation
[Service / model classes]

↓

03 **Data access layer**
DAO interfaces, implementations, SQL mapping
[DAO classes / package]

↓

04 **JDBC and database**
Connections, transactions, constraints and stored data
[DBMS / schema]

ARCHITECTURE RULES

- UI classes contain no SQL
- DAO classes do not control Swing components
- Service layer coordinates use cases
- PreparedStatement for parameterised SQL
- Transactions protect multi-step updates

[Insert project-specific architecture diagram in the PPT if it differs from this baseline]

Actors and system behaviour

Every major functional requirement should appear in at least one use case

ACTORS

01 **[Primary user]**
[Main goal]

02 **[Secondary user]**
[Main goal]

03 **[Administrator / other]**
[Main goal]

Use case	Actor	Purpose	Related FRs
UC-01 [Name]	[Actor]	[Observable user goal]	[FR-__]
UC-02 [Name]	[Actor]	[Observable user goal]	[FR-__]
UC-03 [Name]	[Actor]	[Observable user goal]	[FR-__]
UC-04 [Name]	[Actor]	[Observable user goal]	[FR-__]
UC-05 [Name]	[Actor]	[Observable user goal]	[FR-__]
UC-06 [Name]	[Actor]	[Observable user goal]	[FR-__]

PRESENTATION CHECK

Show the complete use case diagram, then explain two important use cases in detail.

Classes, responsibilities and packages

Demonstrate clear object-oriented decomposition and separation of concerns

INSERT UML CLASS DIAGRAM

[Add classes with essential attributes, major methods, relationships and multiplicities]

UI | Service | Model | DAO

Use meaningful inheritance or interfaces only where behaviour genuinely varies.

Class	Type	Responsibility	Related FRs
[ClassName]	UI / Service / Model / DAO	[One clear responsibility]	[FR-__]
[ClassName]	[Type]	[Responsibility]	[FR-__]
[ClassName]	[Type]	[Responsibility]	[FR-__]
[ClassName]	[Type]	[Responsibility]	[FR-__]

PACKAGE PLAN

[base.package].ui • service • model • dao • util

Entities, relationships and constraints

The schema must support the requirements without unnecessary duplication

INSERT ER DIAGRAM

[Show tables or entities, primary keys, foreign keys and cardinalities]

DATABASE

[DBMS name and version]

Important constraints: [unique / not null / check / FK]

Table	Purpose	Primary key	Foreign key	Related FRs
[table_1]	[Stored information]	[column]	[column → table]	[FR-__]
[table_2]	[Stored information]	[column]	[column → table]	[FR-__]
[table_3]	[Stored information]	[column]	[column → table]	[FR-__]
[table_4]	[Stored information]	[column]	[column → table]	[FR-__]
[table_5]	[Stored information]	[column]	[column → table]	[FR-__]

JDBC DESIGN

PreparedStatement • safe resource closing • transactions where required

Screen navigation and interface design

Each screen should support a defined use case and specify validation and user feedback

NAVIGATION MAP

UI-01 [Login / Home] → UI-02 [Data entry] → UI-03 [Search / View] → UI-04 [Report / Admin]

INSERT ONE REPRESENTATIVE SWING WIREFRAME

[Label important JTextField, JComboBox, JButton, JTable and message areas]

SCREEN REVIEW

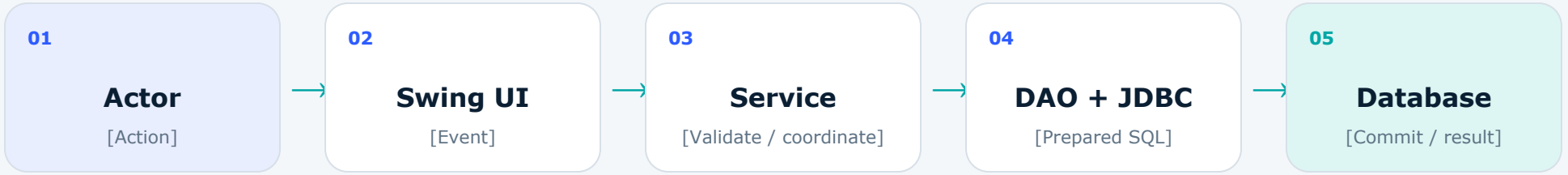
- Actor and use case
- Input components
- Actions and navigation
- Validation messages
- Confirmation / error states
- Related FR IDs
- Keyboard and focus order

Use readable layouts; avoid one oversized JFrame containing every function.

Core sequence, validation and failure handling

Show how one important user action travels from Swing to the database and back

SEQUENCE FOR [UC-__]



VALIDATION

[Required, range, format, duplicate and business rules]

EXCEPTION HANDLING

[Invalid input, database failure, rollback and recovery]

DATA INTEGRITY

[PK, FK, unique, not null and transaction boundaries]

In the document, include at least two complete sequence diagrams for important workflows.

REVIEW SUMMARY

Design traceability and readiness

Confirm that the design is complete enough to begin Code Review 1 implementation

Requirement	Use case	Module / class	Screen	DB table	Verification
FR-01	[UC-__]	[Module / Class]	[UI-__]	[Table]	[Planned test]
FR-02	[UC-__]	[Module / Class]	[UI-__]	[Table]	[Planned test]
FR-03	[UC-__]	[Module / Class]	[UI-__]	[Table]	[Planned test]
FR-04	[UC-__]	[Module / Class]	[UI-__]	[Table]	[Planned test]

IMPLEMENTATION READINESS

- [Team responsibilities assigned]
- [Packages and classes agreed]
- [Database setup planned]
- [Code Review 1 target date]

Team Lead: [Name] | Date: [ ]

DECISION REQUIRED

[Approved / Revise]

Key changes: [_____]

NEXT MILESTONE

Code Review 1

Basic Java + OOP
Coding standards

Carry forward the updated HackerRank table.